МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
„КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ”
НАВЧАЛЬНО-НАУКОВИЙ КОМПЛЕКС
„ІНСТИТУТ ПРИКЛАДНОГО СИСТЕМНОГО АНАЛІЗУ”

ЛАБОРАТОРНА РОБОТА №2

з курсу: «Хмарні технології та сервіси»

Виконав:
студент групи КН-51мп
Сидько Н.С.

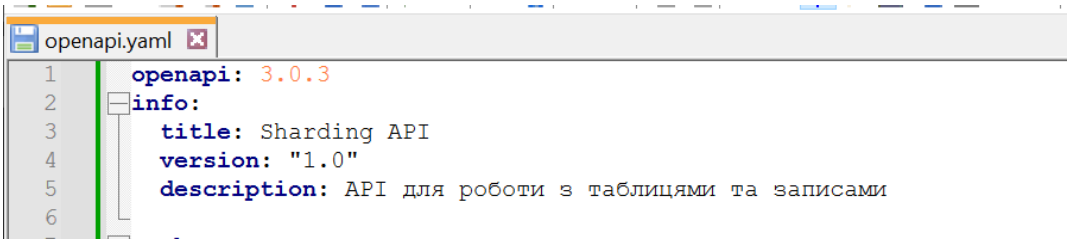
КИЇВ
2025

Хід роботи

1. Create an API specification (OpenAPI REST, GraphQL, gRPC, anything u want) for

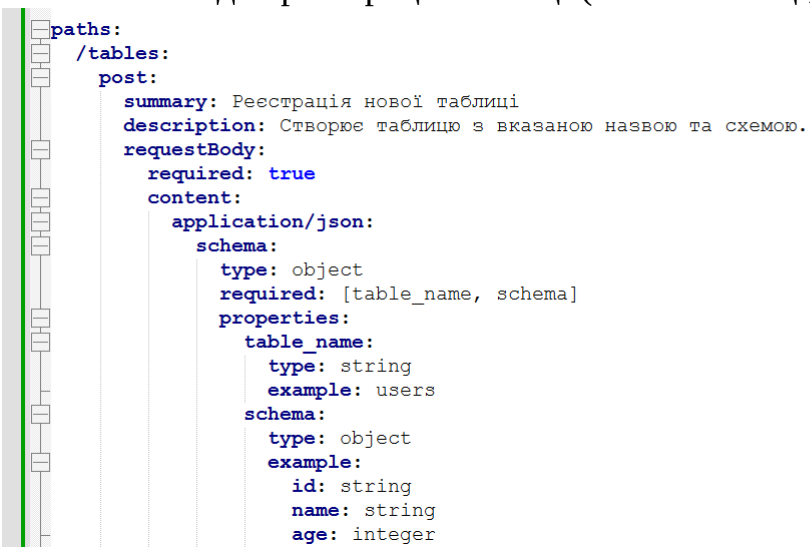
1.1) Registering table definition (1 point)

1. Обрати формат API (наприклад, REST + OpenAPI).



У якості формату API було обрано REST, оскільки він є найбільш поширеним та зручним для інтеграції між сервісами. Для формального опису API використано сдддтандарт OpenAPI 3.0, що дозволяє однозначно визначити структуру запитів і відповідей, а також перевірити коректність специфікації через Swagger Editor.

2. Описати запит для реєстрації таблиці (назва таблиці, схема/поля).



Для реєстрації нової таблиці в специфікації описано ендпоінт POST /tables. Запит приймає JSON-об’єкт, який містить назву таблиці (table_name) та структуру полів (schema).

3. Описати відповідь сервера (успіх / помилка).

```yaml
      responses:
        '200':
          description: Таблицю створено
        '400':
          description: Некоректні дані
```

У відповідях сервера визначено два основні стани:

1) 200 OK - таблицю створено успішно;

2) 400 Bad Request - сервер відхиляє запит через неправильну структуру JSON або відсутні обов’язкові поля.

1.2) CRUD over table records (1 point)

1) Визначити базовий ресурс (наприклад, /tables/{table}/records).

Ресурс для доступу до записів таблиці: /tables/{table}/records
Конкретний запис матиме шлях: /tables/{table}/records/{key}

2) Описати операції:

1. CREATE – створення запису.
2. READ – читання одного або кількох записів.
3. UPDATE – оновлення запису.
4. DELETE – видалення запису.

Для кожної операції: метод (GET/POST/PUT/DELETE), вхідні дані, вихідні дані.

```yaml
  /tables/{table}/records/{key}:
    put:
      summary: Створити або оновити запис
      description: Створює новий запис або оновлює існуючий за ключем.
      parameters:
        - name: table
          in: path
          required: true
          schema:
            type: string
        - name: key
          in: path
          required: true
          schema:
            type: string
      requestBody:
        required: true
        content:
          application/json:
            schema:
              type: object
              properties:
                value:
                  type: object
      responses:
        '200':
          description: Запис створено або оновлено
```

```yaml
get:
  summary: Отримати запис за ключем
  parameters:
    - name: table
      in: path
      required: true
      schema:
        type: string
    - name: key
      in: path
      required: true
      schema:
        type: string
  responses:
    '200':
      description: Запис знайдено
    '404':
      description: Запис не знайдено
```

```yaml
  delete:
    summary: Видалити запис за ключем
    parameters:
      - name: table
        in: path
        required: true
        schema:
          type: string
      - name: key
        in: path
        required: true
        schema:
          type: string
    responses:
      '200':
        description: Запис видалено
      '404':
        description: Запис не знайдено
```

3) (Optional) Make a compound primary key: Partition key + sort key. (2 points)

1. Додати до специфікації поля partition_key і sort_key.

Впровадимо підтримку складеного первинного ключа, який складається з двох окремих частин:
1) partition_key - використовується для групування записів у межах однієї логічної секції (partition);
2) sort_key - визначає конкретний запис всередині partition та дозволяє впорядковувати записи.

У специфікації OpenAPI описані нові параметри partition_key та sort_key, які передаються у шляху ендпоінтів:
/tables/{table}/records/{partition_key}/{sort_key}

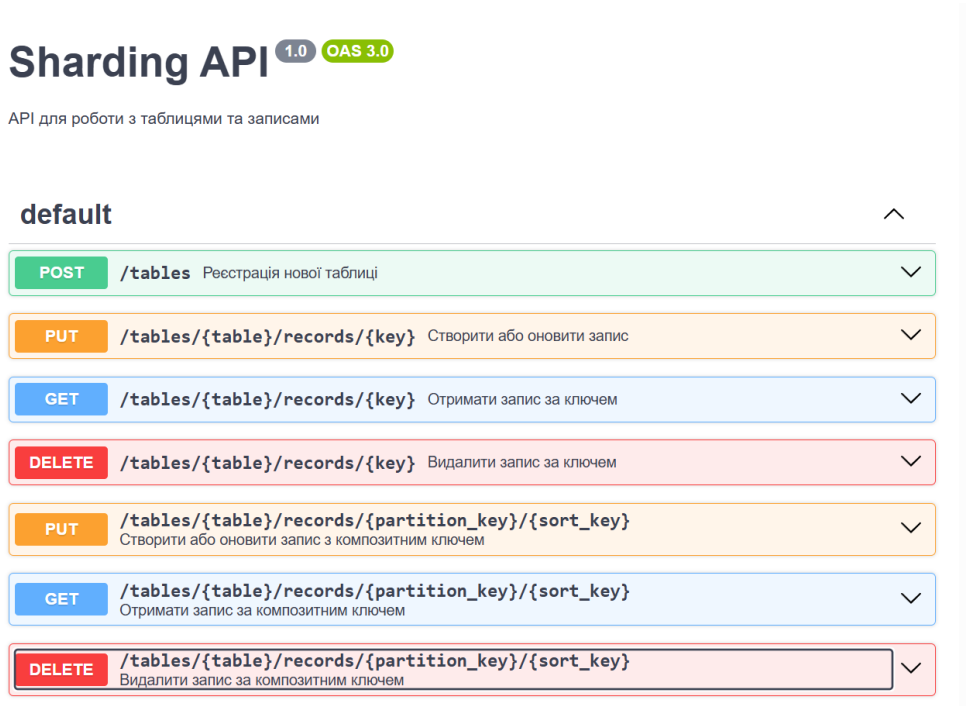
2. Описати, як вони разом формують унікальний ключ запису.

Складений первинний ключ визначає запис у таблиці не одним значенням, а парою значень: (partition_key, sort_key), усі записи з однаковим partition_key зберігаються разом, що полегшує шардінг та прискорює доступ. sort_key у межах одного partition є унікальним і дозволяє однозначно визначити конкретний запис, така структура підтримує можливість зберігати кілька записів, пов'язаних логічно, у межах одного partition. Таким чином, пара ключів працює як повноцінний унікальний ідентифікатор запису.

3. За потреби описати запити, які використовують обидва ключі (наприклад, читання по парі ключів).

Для роботи зі складеним ключем у специфікацію OpenAPI було додано три ендпоінти, що відповідають CRUD-операціям для записів, ідентифікованих парою ключів:

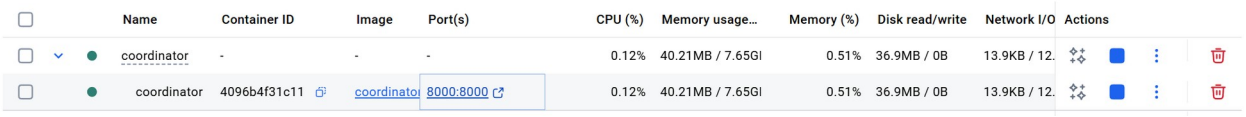
1) PUT /tables/{table}/records/{partition_key}/{sort_key}
2) GET /tables/{table}/records/{partition_key}/{sort_key}
3) DELETE /tables/{table}/records/{partition_key}/{sort_key}



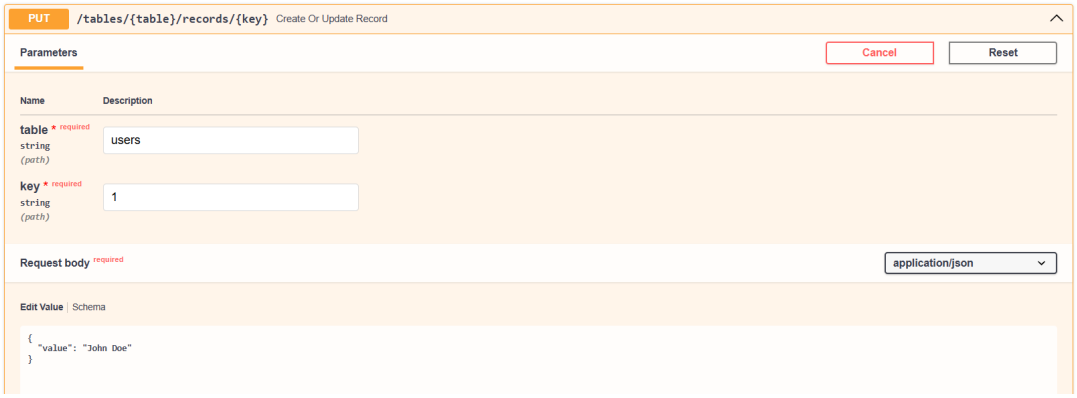
2) Implement the “Coordinator” node which will support this API in single-node mode. Required operations (4 points)
(EXISTS, CREATE, DELETE, READ)

1. Реалізувати один сервіс “Coordinator”, який обробляє запити згідно API зі (1).
2. Додати логіку для операцій:
   EXISTS – перевіряє, чи існує значення за ключем.
   CREATE – створює/записує пару key value.
   DELETE – видаляє значення за ключем.
   READ – повертає значення за ключем.
3. Зберігати дані локально (in-memory / проста локальна БД) — поки ще без шардінгу.

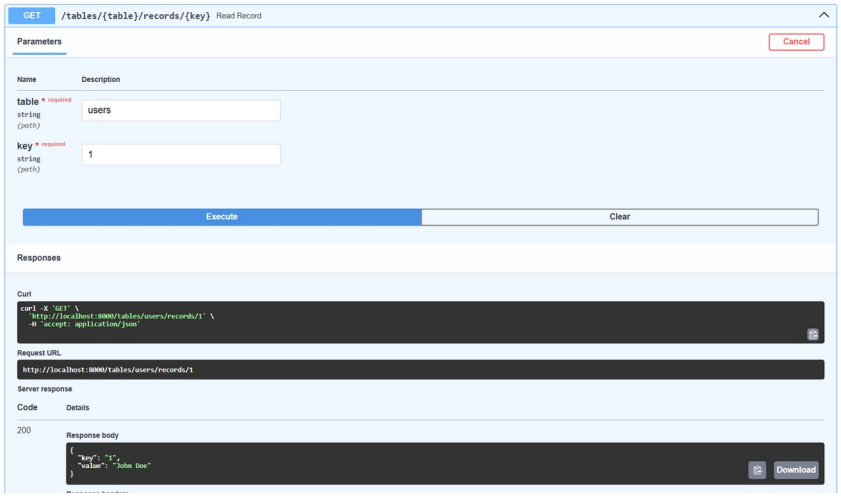
Реалізація сервісу координатор



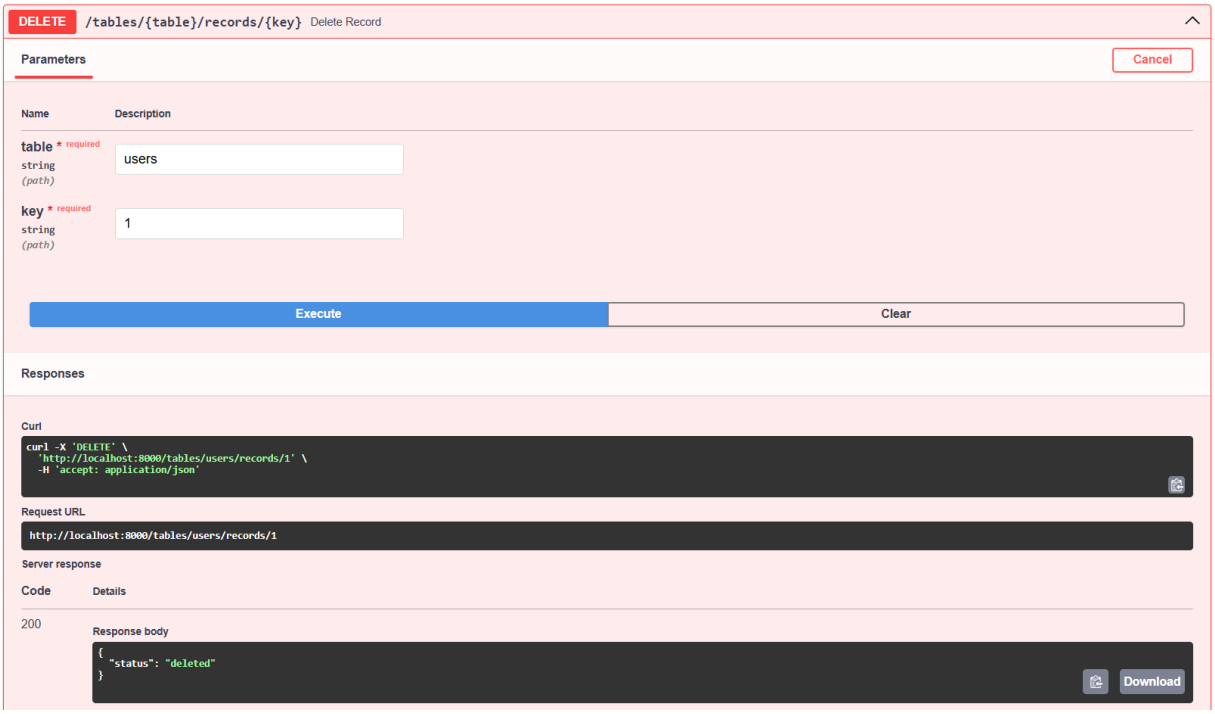
CREATE (PUT) PUT /tables/{table}/records/{key} (створює або оновлює запис - працює)



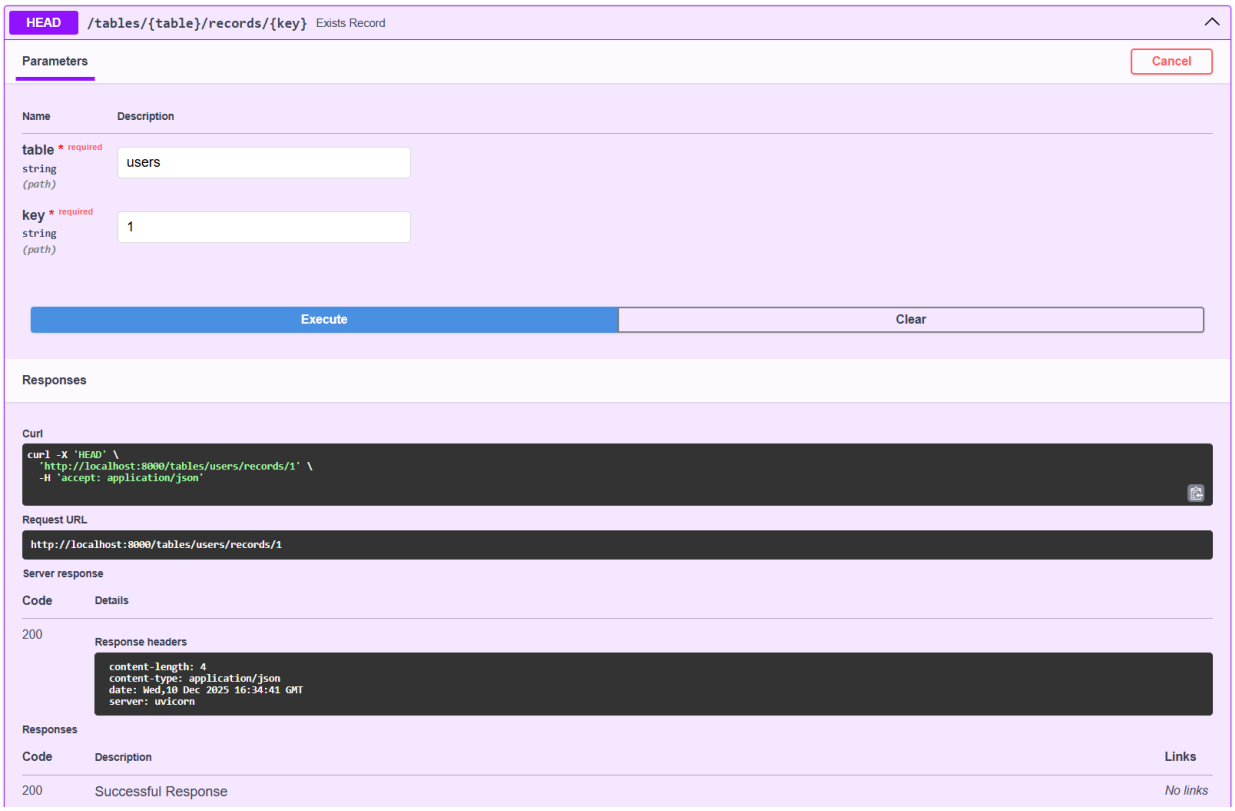
READ (GET) GET /tables/{table}/records/{key} (повертає value – працює)



DELETE (DELETE) DELETE /tables/{table}/records/{key}видаляє – працює



EXISTS (HEAD) HEAD /tables/{table}/records/{key} Повертає 200 або 404 працює.



Будемо зберігати дані локально:

```
tables = {}
compound_tables = {}
```

3) Now Sharding:

a. **Shard writes. Coordinator calculates hash and based on it selects one of downstream shards. Shards are registered in the coordinator upon start. (API needs to be added). Use consistent hashing or similar algorithms for dynamic updates. (3 points)**

Для виконання даного пункту було реалізовано наступне.

Додати API для реєстрації shard-в у Coordinator при старті (наприклад, ендпоінт “зареєструвати шард”).

```
@app.on_event("startup")
def register_shard():
    if not COORDINATOR_URL or not SHARD_ADDRESS:
        print(" SHARD NOT REGISTERED: COORDINATOR
        return
```

```
/shards/register
@router.post("/shards/register")
def register_shard(body: dict):
    address = body["address"]

    if address not in registered_shards:
        registered_shards.append(address)
        hash_ring.add_node(address)

    return {"status": "registered", "shards": registered_shards}
```

Додати структуру для зберігання списку зареєстрованих shard-в у Coordinator.

```
registered_shards = []
```

Реалізувати обчислення хешу від ключа (або partition key).

```
def _hash(self, key: str):  3 usages
    return int(hashlib.md5(key.encode()).hexdigest(), 16)
```

На основі хешу вибирати один з downstream shard-в.

```
def get_node(self, item_key):  1 usage (1 dynamic)
    if not self.ring:
        return None

    hash_val = self._hash(item_key)
    idx = bisect(self.sorted_keys, hash_val) % len(self.sorted_keys)
    return self.ring[self.sorted_keys[idx]]
```

Використати алгоритм (consistent hashing), щоб при додаванні/видаленні shard-в змінювалась мінімальна кількість ключів.

```
def _hash(self, key: str):  3 usages
    return int(hashlib.md5(key.encode()).hexdigest(), 16)

def add_node(self, node):  1 usage (1 dynamic)
    for i in range(self.replicas):
        key = self._hash(f"{node}-{i}")
        self.ring[key] = node
        self.sorted_keys.append(key)
    self.sorted_keys.sort()

def remove_node(self, node):
    for i in range(self.replicas):
        key = self._hash(f"{node}-{i}")
        del self.ring[key]
        self.sorted_keys.remove(key)

def get_node(self, item_key):  1 usage (1 dynamic)
    if not self.ring:
        return None

    hash_val = self._hash(item_key)
    idx = bisect(self.sorted_keys, hash_val) % len(self.sorted_keys)
    return self.ring[self.sorted_keys[idx]]
```

Усі **записи (writes)** (CREATE/DELETE) пересилати з Coordinator на відповідний shard.

```python
def forward(table: str, key: str, method: str, body=None):  4 u
    shard = hash_ring.get_node(key)
    if shard is None:
        raise HTTPException( status_code: 500,  detail: "No shards r

    url = f"{shard}/tables/{table}/records/{key}"

    if method == "PUT":
        r = requests.put(url, json=body)
    elif method == "GET":
        r = requests.get(url)
    elif method == "DELETE":
        r = requests.delete(url)
    elif method == "HEAD":
        r = requests.head(url)
    else:
        raise Exception("Invalid method")

    if r.status_code >= 400:
        raise HTTPException(r.status_code, r.text)

    return r.json() if r.content else {}
```

```python
/tables/{table}/records/{key}
@router.put("/tables/{table}/records/{key}")
def create_or_update_record(table: str, key: str, body: dict):
    return forward(table, key,  method: "PUT", body)

/tables/{table}/records/{key}
@router.delete("/tables/{table}/records/{key}")
def delete_record(table: str, key: str):
    return forward(table, key,  method: "DELETE")
```

b. **Shard reads (2 points)**

1. Для операцій READ/EXISTS Coordinator так само обчислює хеш і визначає відповідний shard.
2. Направляє запит на цей shard і повертає клієнту результат.

```python
/tables/{table}/records/{key}
@router.get("/tables/{table}/records/{key}")
def read_record(table: str, key: str):
    return forward(table, key,  method: "GET")
```

```python
/tables/{table}/records/{key}
@router.head("/tables/{table}/records/{key}")
def exists_record(table: str, key: str):
    return forward(table, key,  method: "HEAD")
```

Результат роботи коду:

Шарди зареєструвались

```
coordinator-1  | INFO:     172.20.0.2:36782 - "POST /shards/register HTTP/1.1" 200 OK
shard1-1       | INFO:     Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
shard2-1       | INFO:     Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
```

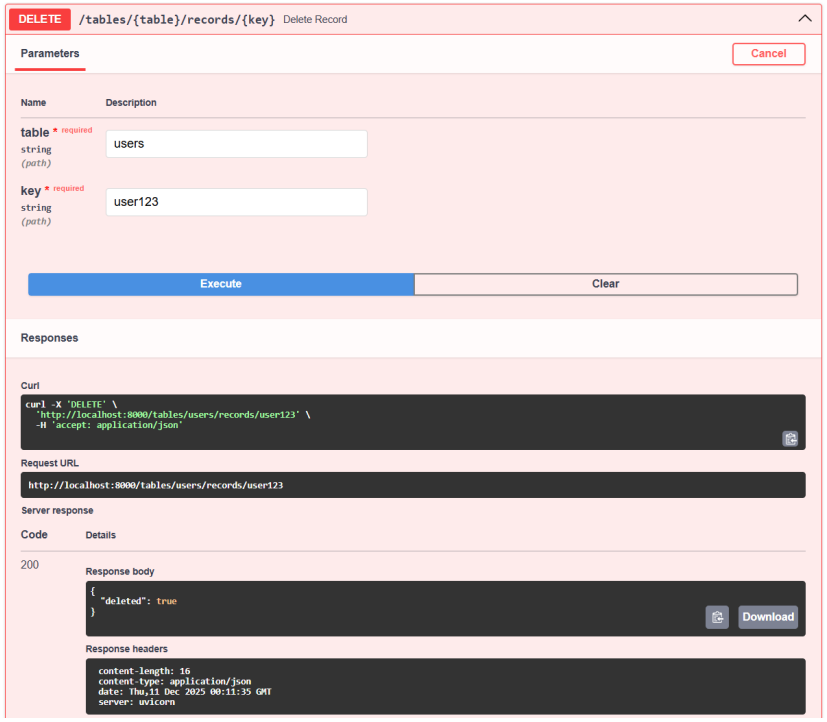
Перевірка shard writes (CREATE/DELETE) через Coordinator

CREATE




```
shard1-1       | SHARD REGISTERED: http://shard1:8000
shard1-1       | INFO:     172.20.0.4:59432 - "PUT /tables/users/records/user123 HTTP/1.1" 200 OK
```

DELETE



Перевірка shard reads (READ/EXISTS) через Coordinator

```
shard2-1       | INFO:     172.20.0.4:55876 - "GET /tables/user999/records/user999 HTTP/1.1" 200 OK
coordinator-1  | INFO:     172.20.0.1:51214 - "GET /tables/user999/records/user999 HTTP/1.1" 200 OK
```

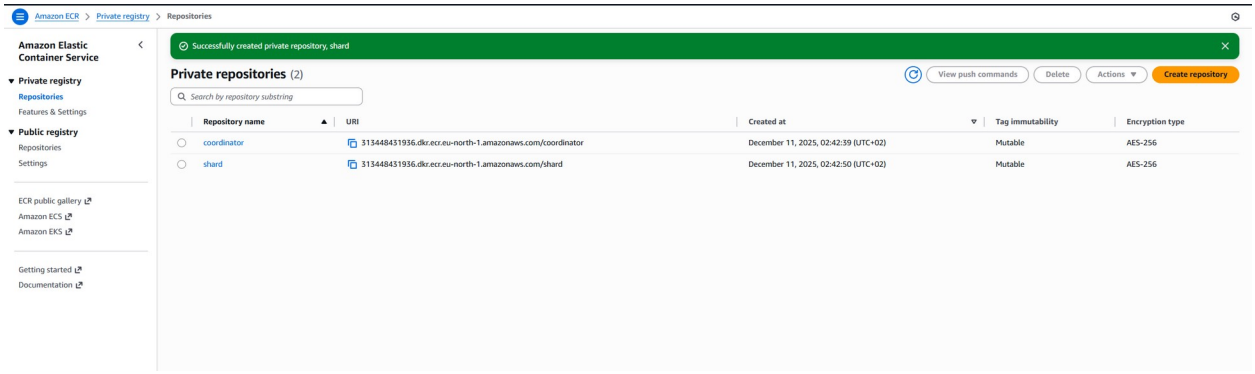
```
shard1-1       | INFO:     172.20.0.3:33230 - "GET /tables/users/records/user123 HTTP/1.1" 200 OK
coordinator-1  | INFO:     172.20.0.1:37874 - "GET /tables/users/records/user123 HTTP/1.1" 200 OK
shard1-1       | INFO:     172.20.0.3:57920 - "HEAD /tables/users/records/user123 HTTP/1.1" 200 OK
coordinator-1  | INFO:     172.20.0.1:48022 - "HEAD /tables/users/records/user123 HTTP/1.1" 200 OK
```

4) Define your infra with IaC (2 points)

1. Обрати інструмент IaC (наприклад, Terraform або CloudFormation).
2. Описати інфраструктуру для розгортання Coordinator та shard-в (обчислювальні ресурси, мережа тощо).
3. Забезпечити можливість розгорнути всю систему через IaC-конфігурацію.

На цьому етапі ми перейшли до розгортання всієї інфраструктури у хмарі AWS. Спочатку ми виконали базову підготовку: увійшли в AWS Console, переконалися, що Docker встановлений локально, та створили приватні репозиторії в Amazon ECR для двох сервісів coordinator та shard. У ці репозиторії були завантажені Docker-образи застосунків, щоб ECS Fargate міг отримувати їх при запуску контейнерів.

```
PS C:\Users\user> aws ecr get-login-password --region eu-north-1 | docker login --username AWS --password-stdin 313448431936.dkr.ecr.eu-north-1.amazonaws.com
>>
Login Succeeded
PS C:\Users\user> cd C:\Users\user\PycharmProjects\coordinator
>>
PS C:\Users\user\PycharmProjects\coordinator> aws ecr get-login-password --region eu-north-1 | docker login --username AWS --password-stdin 313448431936.dkr.ecr.eu-north-1.amazonaws.com
>>
Login Succeeded
PS C:\Users\user\PycharmProjects\coordinator> docker build -t cloud-lab-image .
>>
[+] Building 1.7s (11/11) FINISHED                                                                  docker:desktop-linux
 => [internal] load build definition from Dockerfile                                                                0.0s
 => => transferring dockerfile: 289B                                                                                0.0s
 => [internal] load metadata for docker.io/library/python:3.11-slim                                                 1.1s
 => [auth] library/python:pull token for registry-1.docker.io                                                       0.0s
 => [internal] load .dockerignore                                                                                   0.0s
 => => transferring context: 2B                                                                                     0.0s
 => [1/5] FROM docker.io/library/python:3.11-slim@sha256:7cd0079a9bd8800c81632d65251048fc2848bf9afda542224b1b10e0   0.1s
 => => resolve docker.io/library/python:3.11-slim@sha256:7cd0079a9bd8800c81632d65251048fc2848bf9afda542224b1b10e0   0.1s
 => [internal] load build context                                                                                   0.1s
 => => transferring context: 214.35kB                                                                               0.1s
 => CACHED [2/5] WORKDIR /app                                                                                       0.0s
 => CACHED [3/5] COPY requirements.txt .                                                                            0.0s
 => CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt                                                 0.0s
 => CACHED [5/5] COPY . .                                                                                           0.0s
 => exporting to image                                                                                              0.2s
 => => exporting layers                                                                                             0.0s
 => => exporting manifest sha256:d6640dcfbb875659e254ddd4bd99d5b6f1d2086b8be02d829f5526cc321836d1                   0.0s
 => => exporting config sha256:0077f2a347aaeff794bfec833fa8c4aef64bf0a42a04c0429f984b6fdf97f933                     0.0s
 => => exporting attestation manifest sha256:155b3b9c9c65a17c3de3b792b4d699ad269abc1e48ae4ce390c33759d150a241       0.1s
 => => exporting manifest list sha256:124f1382cd2fdbd75c5eca6fc55952b33d6ead702195d6b8cd497e3ca1f30a81              0.0s
 => => naming to docker.io/library/cloud-lab-image:latest                                                           0.0s
 => => unpacking to docker.io/library/cloud-lab-image:latest                                                        0.0s
PS C:\Users\user\PycharmProjects\coordinator> docker tag cloud-lab-image:latest 313448431936.dkr.ecr.eu-north-1.amazonaws.com/coordinator:latest
PS C:\Users\user\PycharmProjects\coordinator> docker tag cloud-lab-image:latest 313448431936.dkr.ecr.eu-north-1.amazonaws.com/shard:latest
PS C:\Users\user\PycharmProjects\coordinator> docker push 313448431936.dkr.ecr.eu-north-1.amazonaws.com/coordinator:latest
The push refers to repository [313448431936.dkr.ecr.eu-north-1.amazonaws.com/coordinator]
f7304492cba6: Pushed
1733a4cd5954: Pushed
3f0cdbca744e: Pushed
72cf4c3b8301: Pushed
732225d44a55: Pushed
68f110696e60: Pushed
100c1db52aad: Pushed
324b2d5264ee: Pushed
4d55cfecf366: Pushed
latest: digest: sha256:124f1382cd2fdbd75c5eca6fc55952b33d6ead702195d6b8cd497e3ca1f30a81 size: 856
PS C:\Users\user\PycharmProjects\coordinator> docker push 313448431936.dkr.ecr.eu-north-1.amazonaws.com/shard:latest
The push refers to repository [313448431936.dkr.ecr.eu-north-1.amazonaws.com/shard]
3f0cdbca744e: Pushed
732225d44a55: Pushed
f7304492cba6: Pushed
4d55cfecf366: Pushed
324b2d5264ee: Pushed
68f110696e60: Pushed
1733a4cd5954: Pushed
72cf4c3b8301: Pushed
100c1db52aad: Pushed
latest: digest: sha256:124f1382cd2fdbd75c5eca6fc55952b33d6ead702195d6b8cd497e3ca1f30a81 size: 856
PS C:\Users\user\PycharmProjects\coordinator>
```



Далі був створений спеціальний CloudFormation YAML-файл, у якому ми описали всю необхідну інфраструктуру. У шаблоні були визначені такі ресурси, як VPC, дві публічні підмережі, інтернет-шлюз, таблиця маршрутизації, група безпеки для доступу до сервісів, ECS-кластер, ролі IAM, а також два сервіси ECS: CoordinatorService та ShardService. Ми також додали параметри, що дозволили передати посилання на образи ECR та кількість шардових контейнерів.

FileForLab2.yaml

```yaml
AWSTemplateFormatVersion: "2010-09-09"
Description: "Cloud Lab 2 — Coordinator + Shards on ECS Fargate (eu-north-1)"

Parameters:
  CoordinatorImage:
    Type: String
    Default: "313448431936.dkr.ecr.eu-north-1.amazonaws.com/coordinator:latest"
    Description: "ECR image URI for coordinator"

  ShardImage:
    Type: String
    Default: "313448431936.dkr.ecr.eu-north-1.amazonaws.com/shard:latest"
    Description: "ECR image URI for shard"

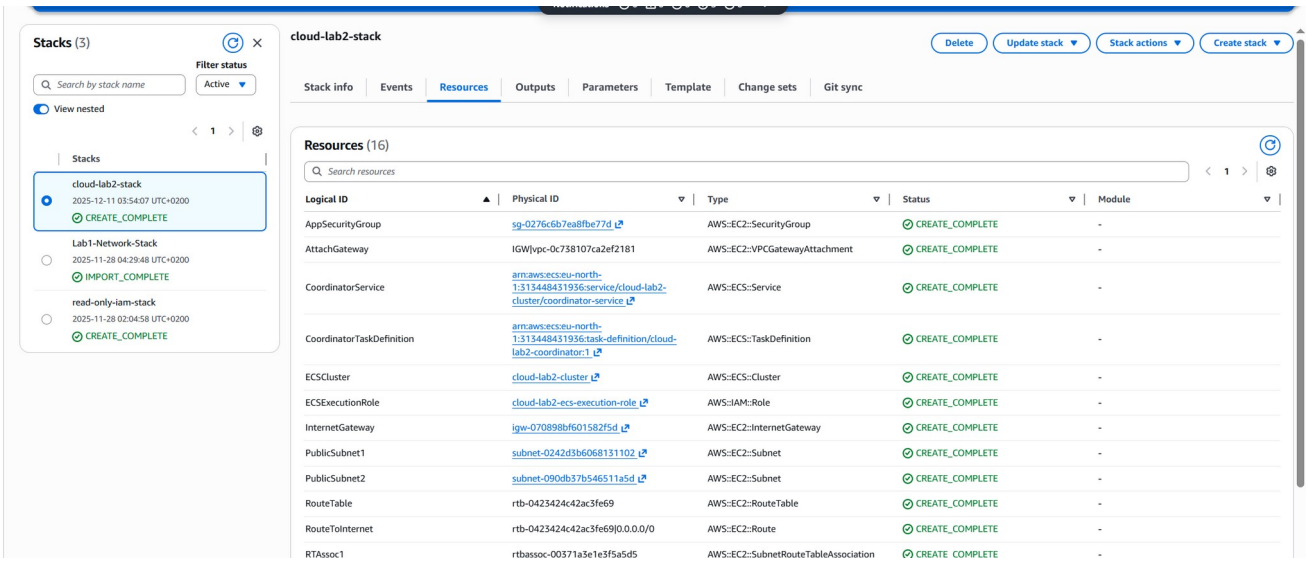
  DesiredShards:
    Type: Number
    Default: 3
    MinValue: 1
    MaxValue: 10
    Description: "How many shard tasks to run"


Resources:
  VPC:
    Type: AWS::EC2::VPC
    Properties:
      CidrBlock: "10.0.0.0/16"
      EnableDnsSupport: true
      EnableDnsHostnames: true
      Tags:
        - Key: Name
          Value: "cloud-lab2-vpc"

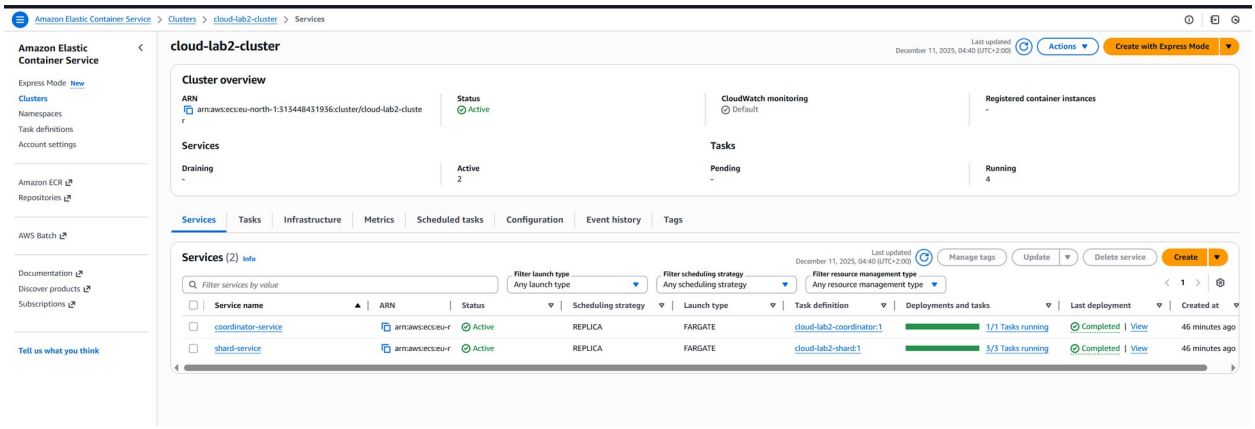
  InternetGateway:
    Type: AWS::EC2::InternetGateway
    Properties:
      Tags:
        - Key: Name
          Value: "cloud-lab2-igw"

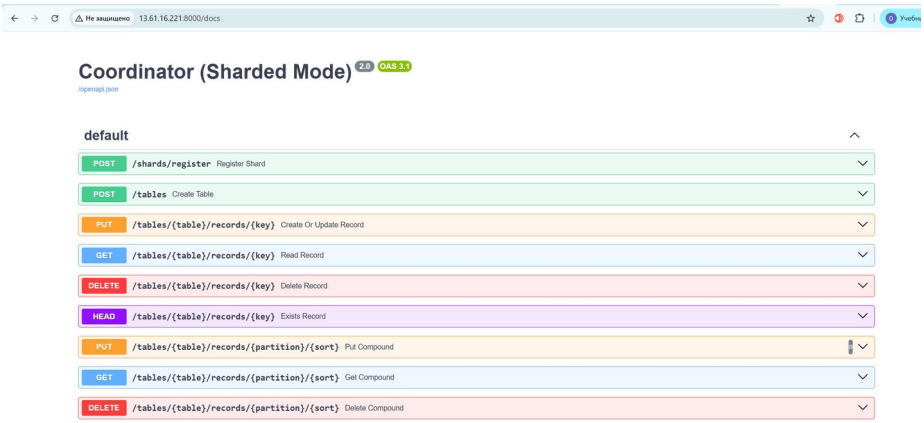
  AttachGateway:
```

Після завантаження цього YAML-файлу в AWS CloudFormation був створений стек, який автоматично розгорнув усі вказані ресурси. Кожен компонент від мережевої інфраструктури до контейнери Fargate був налаштований і запущений без необхідності ручної конфігурації.



У результаті ми отримали повністю функціональну хмарну систему з координатором і кількома шардовими контейнерами. Кожен з них має власну публічну IP-адресу, взаємодіє через VPC та працює в режимі реплікації. Coordinator успішно запускається і надає API, доступне через браузер за публічною адресою порт 8000. Таким чином, пункт 4 повністю виконує вимогу розгорнути мультиконтейнерну архітектуру у AWS за допомогою інфраструктури як коду.





Висновок

У ході роботи було створено повноцінну систему горизонтального шардінгу з координатором, декількома шардами та механізмом розподілу даних на основі узгодженого хешування. Реалізовано API для створення таблиць і виконання операцій створення, читання, оновлення та видалення записів. Система автоматично реєструє шард вузли та рівномірно розподіляє ключі між ними. Додатково впроваджено підтримку складеного ключа, який складається з двох частин, що дозволяє зберігати упорядковані записи в межах однієї логічної секції. Для хмарного розгортання була застосована інфраструктура як код, завдяки чому створено VPC, мережеві ресурси та кластер ECS Fargate. У результаті створено повністю функціональну систему масштабованого розподіленого зберігання даних.

Контрольні питання

1) Types of scalability and when to use each

Існує вертикальне та горизонтальне масштабування. Вертикальне масштабування означає збільшення ресурсів одного сервера. Його варто використовувати коли система не вимагає великої кількості вузлів або коли логіка погано розбивається на частини. Горизонтальне масштабування означає додавання нових серверів і підходить у випадках великої кількості запитів, великих обсягів даних або необхідності високої відмовостійкості.

2) Types of sharding and their use cases

Існує кілька видів шардінгу. Статичний шардінг коли діапазони ключів розподілені вручну, підходить для систем з передбачуваними обсягами даних. Динамічний шардінг коли система сама визначає розподіл, добре працює при нерівномірному навантаженні. Географічний шардінг використовується коли дані потрібно розміщувати ближче до користувачів для прискорення відповіді. Узгоджене хешування підходить для систем де шарди можуть додаватися та видалятися.

3) Consistent hashing algorithm

Узгоджене хешування створює віртуальне кільце з хеш значень. Кожен шард отримує багато віртуальних позицій на кільці. Ключ також хешується та розміщується на кільці. Ключ зберігається у першому шард вузлі який знаходиться після його позиції на кільці. Такий підхід дозволяє додавати або видаляти шарди так щоб переміщалась лише невелика частина ключів.

4) How a HashMap works

Хеш мапа використовує хеш функцію для обчислення індексу у внутрішньому масиві. Якщо два ключі потрапляють у той самий індекс виникає колізія. Для цього використовується ланцюжкове зберігання або деревоподібні структури. Основні операції мають середню складність пошуку та вставки що дорівнює одній операції.

5) Why partition key and sort key are needed

Партішн ключ визначає у якій логічній секції таблиці зберігається група даних. Сорт ключ визначає порядок елементів всередині цієї секції. Разом вони створюють складений ключ який дозволяє швидко знаходити конкретний запис або набір записів у заданому діапазоні в межах однієї групи.

6) Linearization and serialization in data consistency

Лінеаризація означає що всі операції над даними виглядають так ніби вони виконуються у чіткій послідовності навіть якщо система є розподіленою. Серіалізація означає що одночасні операції виконуються у визначеному порядку для уникнення конфліктів. Обидва підходи забезпечують узгодженість даних коли кілька вузлів виконують зміну одного і того ж стану.

Додаток 1

Main.py

```
from fastapi import FastAPI

from app.tables import router as tables_router
from app.records import router as records_router
from app.records_compound import router as compound_router
from app.shards import router as shards_router

app = FastAPI(
    title="Coordinator (Sharded Mode)",
    version="2.0"
)

app.include_router(shards_router)
app.include_router(tables_router)
app.include_router(records_router)
app.include_router(compound_router)
```

memory.py

```
tables = {}
compound_tables = {}
registered_shards = []
hash_ring = None
```

records.py

```python
from fastapi import APIRouter, HTTPException
import requests

from app.sharding import hash_ring

router = APIRouter()

def forward(table: str, key: str, method: str, body=None):
    shard = hash_ring.get_node(key)
    if shard is None:
        raise HTTPException(500, "No shards registered")

    url = f"{shard}/tables/{table}/records/{key}"

    if method == "PUT":
        r = requests.put(url, json=body)
    elif method == "GET":
        r = requests.get(url)
    elif method == "DELETE":
        r = requests.delete(url)
    elif method == "HEAD":
        r = requests.head(url)
    else:
        raise Exception("Invalid method")

    if r.status_code >= 400:
        raise HTTPException(r.status_code, r.text)

    return r.json() if r.content else {}


@router.put("/tables/{table}/records/{key}")
def create_or_update_record(table: str, key: str, body: dict):
    return forward(table, key, "PUT", body)


@router.get("/tables/{table}/records/{key}")
def read_record(table: str, key: str):
    return forward(table, key, "GET")


@router.delete("/tables/{table}/records/{key}")
def delete_record(table: str, key: str):
    return forward(table, key, "DELETE")


@router.head("/tables/{table}/records/{key}")
def exists_record(table: str, key: str):
    return forward(table, key, "HEAD")
```

records_compound.py

```python
from fastapi import APIRouter, HTTPException
from app.memory import compound_tables

router = APIRouter()

@router.put("/tables/{table}/records/{partition}/{sort}")
def put_compound(table, partition, sort, body):
    if table not in compound_tables:
        raise HTTPException(404, "Table not found")
```

```
    compound_tables[table].setdefault(partition, {})
    compound_tables[table][partition][sort] = body["value"]
    return {"status": "ok"}

@router.get("/tables/{table}/records/{partition}/{sort}")
def get_compound(table, partition, sort):
    if (
        table not in compound_tables
        or partition not in compound_tables[table]
        or sort not in compound_tables[table][partition]
    ):
        raise HTTPException(404, "Record not found")

    return {
        "partition_key": partition,
        "sort_key": sort,
        "value": compound_tables[table][partition][sort]
    }

@router.delete("/tables/{table}/records/{partition}/{sort}")
def delete_compound(table, partition, sort):
    if (
        table not in compound_tables
        or partition not in compound_tables[table]
        or sort not in compound_tables[table][partition]
    ):
        raise HTTPException(404, "Record not found")
    del compound_tables[table][partition][sort]
    return {"status": "deleted"}
```

runner.py

```
import os
import uvicorn

mode = os.getenv("MODE", "coordinator")

if mode == "coordinator":
    uvicorn.run("app.main:app", host="0.0.0.0", port=8000)
else:
    uvicorn.run("app.shard_app:app", host="0.0.0.0", port=8000)
```

shard_app.py

```
import os
import requests
from fastapi import FastAPI, HTTPException, Request
import time
app = FastAPI(title="Shard Node")


storage = {}


COORDINATOR_URL = os.getenv("COORDINATOR_URL")
SHARD_ADDRESS = os.getenv("SHARD_ADDRESS")


@app.on_event("startup")
def register_shard():
    """
    Автоматично реєструє shard у Coordinator.
    Якщо Coordinator ще не запустився — пробує повторно.
    """
```

```
    if not COORDINATOR_URL or not SHARD_ADDRESS:
        print("SHARD NOT REGISTERED: COORDINATOR_URL або SHARD_ADDRESS не
задані.")
        return

    for attempt in range(20):
        try:
            print(f" Registering shard at {COORDINATOR_URL} (attempt
{attempt+1}) ...")
            r = requests.post(
                f"{COORDINATOR_URL}/shards/register",
                json={"address": SHARD_ADDRESS},
                timeout=3
            )
            r.raise_for_status()
            print("SHARD REGISTERED:", SHARD_ADDRESS)
            return
        except Exception as ex:
            print(f"Coordinator not ready yet ({ex}). Retrying in 2 sec...")
            time.sleep(2)

    print("FAILED TO REGISTER SHARD AFTER 20 ATTEMPTS")



@app.put("/tables/{table}/records/{key}")
async def put_record(table: str, key: str, request: Request):
    body = await request.json()
    value = body.get("value")

    table_ref = storage.setdefault(table, {})
    table_ref[key] = value

    return {"stored": True, "table": table, "key": key, "value": value}


@app.get("/tables/{table}/records/{key}")
def get_record(table: str, key: str):
    if table not in storage or key not in storage[table]:
        raise HTTPException(404, "Not found")

    return {"value": storage[table][key]}


@app.delete("/tables/{table}/records/{key}")
def delete_record(table: str, key: str):
    if table in storage and key in storage[table]:
        del storage[table][key]

    return {"deleted": True}


@app.head("/tables/{table}/records/{key}")
def exists_record(table: str, key: str):
    if table in storage and key in storage[table]:
        return ""
    raise HTTPException(404, "Not found")
```

sharding.py

```
import hashlib
from bisect import bisect

class ConsistentHashRing:
```

```
    def __init__(self, replicas=100):
        self.replicas = replicas
        self.ring = {}
        self.sorted_keys = []

    def _hash(self, key: str):
        return int(hashlib.md5(key.encode()).hexdigest(), 16)

    def add_node(self, node):
        for i in range(self.replicas):
            key = self._hash(f"{node}-{i}")
            self.ring[key] = node
            self.sorted_keys.append(key)
        self.sorted_keys.sort()

    def remove_node(self, node):
        for i in range(self.replicas):
            key = self._hash(f"{node}-{i}")
            del self.ring[key]
            self.sorted_keys.remove(key)

    def get_node(self, item_key):
        if not self.ring:
            return None

        hash_val = self._hash(item_key)
        idx = bisect(self.sorted_keys, hash_val) % len(self.sorted_keys)
        return self.ring[self.sorted_keys[idx]]


hash_ring = ConsistentHashRing()
```

shards.py

```
from fastapi import APIRouter
from app.memory import registered_shards
from app.sharding import hash_ring

router = APIRouter()

@router.post("/shards/register")
def register_shard(body: dict):
    address = body["address"]

    if address not in registered_shards:
        registered_shards.append(address)
        hash_ring.add_node(address)

    return {"status": "registered", "shards": registered_shards}
```

tables.py

```
from fastapi import APIRouter, HTTPException
from app.memory import tables, compound_tables

router = APIRouter()

@router.post("/tables")
def create_table(body: dict):
    name = body["table_name"]

    if name in tables:
        raise HTTPException(400, "Table already exists")
```

```
    tables[name] = {}
    compound_tables[name] = {}

    return {"status": "table_created", "table": name}
```